SmartStride (The Future of Mobility: Hands-Free, Affordable, and Intelligent)

Matthew Itskovich, Evan Reese, Arturo Lara, and Adam Lilly

Dept. of Electrical Engineering and Computer Science, University of Central Florida, Orlando, Florida, 32816-2450

Abstract – SmartStride **is an affordable, hands-free mobility system designed to retrofit existing electric wheelchairs with autonomous capabilities. The system integrates a Raspberry Pi 5-based control unit that processes data from a magnetometer and infrared depth sensor (Kinect V2) to enable real-time obstacle detection and navigation. Five custom PCBs support the design: three voltage regulator boards using LM2596 and AMS1117 chips, a Bluetooth communication controller, and a Pi-to-Wheelchair interface board. Together, these modules create an accessible, low-cost platform for intelligent wheelchair automation.**

I. INTRODUCTION

Mobility impairment continues to burden and limit millions of individuals all over the world who rely on wheelchairs for daily movement. While advanced electric wheelchairs designs do exist to help with this issue, their manufacturing costs and the effort to rebuild the complex design prevents widespread accessibility [1]. To address this issue, SmartStride introduces an affordable, hands-free mobility system that updates existing electric wheelchairs with autonomous navigation capabilities.

The system uses Raspberry Pi 5 as the primary control unit to take in sensor readings and implement movement commands. A Kinect V2 IR sensor with depth mapping capabilities helps the user with spatial awareness and obstacle detection, while the magnometer that is integrated keeps track of the orientation of the wheelchair so it knows where it is in a given setting. Five custom-designed printed circuit boards support the system functionality: Three of these are voltage regulator boards utilizing the LM2596 and AMS 1117 chips, a Bluetooth communication controller for wireless operation, and a pi-to-wheelchair interface for power and signal distribution.

The goal of this project is to improve the mobility and overall safety of users with limited body control through intelligent motion assistance, all while maintaining an affordable cost and easy to implement modular design.

II. SYSTEM OVERVIEW

The SmartStride system is designed around two primary computing units that work together to achieve both manual and autonomous control of the wheelchair. The first computing unit is a wireless controller built around an ESP32 microcontroller. The second is a Raspberry Pi 5 mounted on the wheelchair chassis, responsible for interpreting control signals and coordinating sensor inputs for motion control.

In manual mode, the ESP32 reads analog voltage signals from a joystick using its built-in analog-to-digital converter (ADC). These readings represent the user's directional input and are transmitted wirelessly via Bluetooth to the Raspberry Pi. The Pi receives the data, interprets it, and translates it into digital values that are sent to a digital-to-analog converter (DAC) through the I²C communication protocol. The DAC converts these digital instructions into analog voltages—typically between 1 V and 5 V—that emulate the original joystick behavior. The wheelchair responds to these signals exactly as it would to the stock controller. To ensure safe startup conditions, the DAC outputs a default resting voltage of 2.5 V on all four control lines, representing a neutral "joystick at rest" state.

In autonomous mode, the Pi processes real-time depth data captured by the Kinect V2 sensor to construct a spatial map of the surrounding environment. By analyzing object distance within the depth field, the system determines when obstacles are present. If an obstacle is detected, the Pi overrides the DAC output and commands the wheelchair to stop. Otherwise, it maintains a steady forward profile based on predefined navigation parameters.

Speed is governed by a fixed potentiometer value to ensure predictable and safe operation during testing. Overall, Smart Stride's dual-processor architecture allows seamless switching between manual and autonomous control, combining low-latency wireless input with robust sensor-driven navigation for enhanced mobility and safety. This type of IR sensor has a history of use in research and experimental settings.

III. REVERSE ENGINEERING

When we were first given our electric wheelchair, it was nonfunctional and we had limited knowledge about its operation. We were unable to find definitive proof of what model wheelchair this was, but we were able to find the model numbers for various parts, such as the joystick and

batteries of the wheelchair. We needed to replace the batteries on the wheelchair, which were 2-12V Lead Acid batteries, and regain operation of the wheelchair. After we were able to repair the wheelchair, we then moved to the controller board on the wheelchair.

After disassembly, we decided to remove the joystick from the wheelchair, and then measure the voltage along the joystick's data lines to learn how it operated. We reached out to the joystick's manufacturer, who no longer had information on the model of joystick we had, and the joystick used an atypical connection with 4 data wires rather than the industry standard 2. We learned that each pair of wires shared an inverse direct relationship with another, so for example data lines 1 would be 2V and line 2 would be 3V to sum to 5V. The output of the joystick was analog so it would change between 1V-4V depending on the position of the joystick. Using this knowledge we acquired a DAC that had 4 analog outputs and programmed it with profiles similar to the joystick. All data lines would be 2.5V when neutral, and would need to be neutral to allow for the wheelchair to start up. Without the joystick being neutral, the wheelchair would not turn on. If we don't maintain the ratio between the 4 data lines the wheelchair would simply stop.

It is imperative we maintain the 5V ratios so ensure consistent operation and allow for us to control the wheelchair. We decided to reuse the joystick in our wireless controller design. Since it is a potentiometer, it would give the same ratio just at 3.3V, so we can then simply translate that ratio to the DAC for 5V operation when receiving from the wireless controller.

IV. HARDWARE DESIGN

A. Hardware Block Diagram:

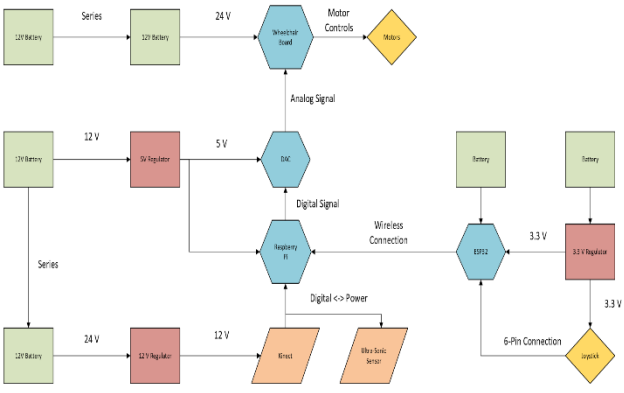

Fig. 1. Overall hardware block diagram of the SmartStride system

The above hardware flow chart has 4 total 12V batteries. Two 20 Ah feeding the two motors controlling the back wheels and wheelchair control unit board with the joystick. The other two 12 V batteries feed each of the regulator PCBs which then feed every other component we added on to the chair. The 5V regulator feeds off one of the 12V batteries. From the 5V regulator, the raspberry pi and DAC all take power. The Raspberry Pi operates between 3A and 5 A. The DAC feeds off it around 0.4 mA. The 3.3V regulator is hooked up to the controller board PCB that is fed by two AA-batteries. The controller board also has the LED display and joystick hooked up to it.

B. PCB DESIGN

For the SmartStride system, we designed five custom PCBs that handle power, control, and communication across the wheelchair. Three of these boards are voltage regulators (12 V, 5 V, and 3.3 V) used to supply stable power to components like the Raspberry Pi, Kinect, and sensors. The other two are the Pi-to-Wheelchair interface board and the ESP32 controller board, which manage wired and wireless communication. All designs were created in Fusion 360 and ordered through JLCPCB for testing and integration

1) 5V and 12 V regulator

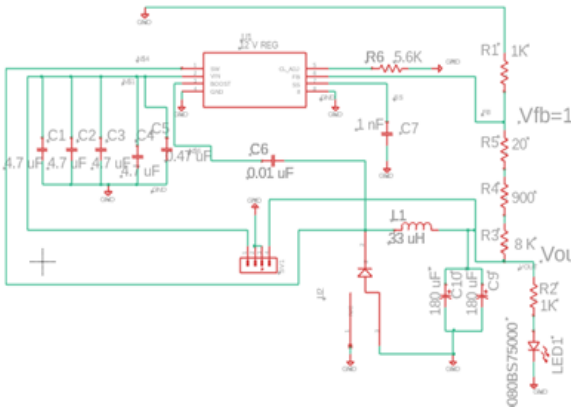

Fig. 2. 5V and 12 V LM2596-ADJ regulator PCB schematic

2)3.3V Regulator

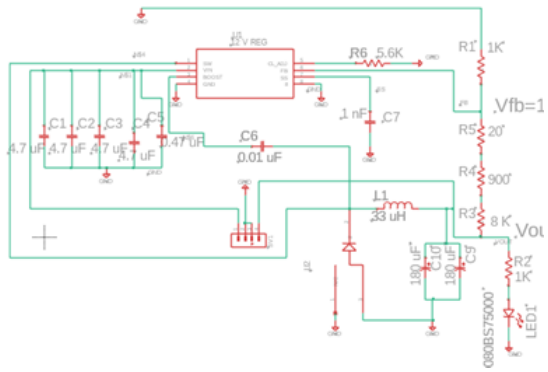

Fig. 3. 3.3V AMS 117regulator PCB schematic

The 3.3V regulator takes an input voltage ranging from 4.75V -10V and outputs a maximum current of 1A [2]. The regulator chip used for this PCB is the AMS 117. In the schematic, it has a feedback loop, output capacitor to stabilize the feedback voltage, and resistor and LED in series also hooked up to the same node as a 22 uF capacitor.

3)ESP32 Controller Board

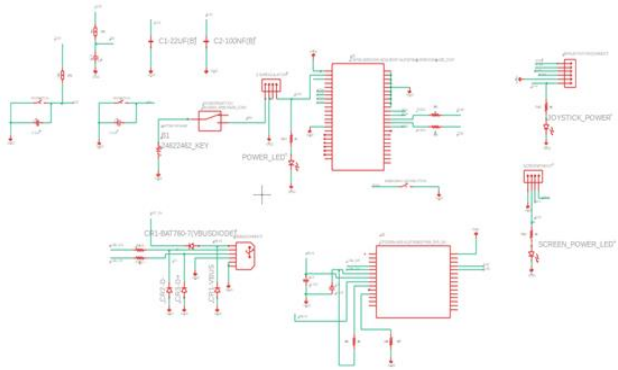

Fig. 3. ESP32 Controller Board Schematic

Looking at this schematic, we can see the main ESP32 chip and its pinout. A four-pin header on the right side connects to the joystick, while another four-pin header interfaces with the display module. There is also a USB pinout that allows data to be passed from the Raspberry Pi to the ESP32 for communication. Additionally, the schematic shows a four-pin connection between the ESP32 and the 3.3 V regulator, supplying power to the controller and its peripherals.

4)Pi to Wheelchair PCB Schematic

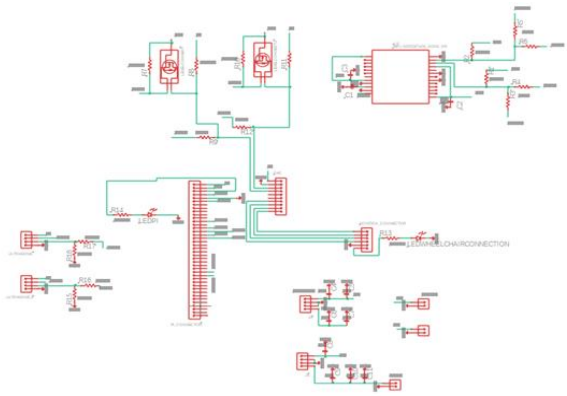

Fig. 5. Pi to wheelchair schematic

The Raspberry Pi connects through a 40-pin header, where Pin 1 receives 5 V from the battery via the regulator and Pin 2 verifies power flow through an LED. Pins 5 (SDA) and 7 (SCL) form the I²C bus for communication with the DAC, using a level converter to match the 5 V logic. Two additional I²C lines link the Pi to the MPU for sensor data transfer, and several GPIO pins handle interrupt signals and general control. Two 4-pin headers connect to external sensors, the magnetometer on the rear wheel for orientation and the ultrasonic sensor at the base for obstacle detection. The MPU reports acceleration and rotation data, while level converters ensure all Pi-to-DAC communication remains voltage-compatible.

V. SYSTEM COMPONENTS

A. *Kinect V2 Sensor*

The eyes of this project is the Kinect. This sensor array has 2 high quality sensors: the first type is a camera with a 1080p resolution, is RGB, and registers at 30 fps which makes for smooth and quality recordings. The second type of sensor is an IR depth camera that has 512x424 p and also has a smoothness of 30 fps. The depth camera will be the primary used sensor out of the two. The IR depth range is about 0.5 to 4.5 meters. Also, the IR also has a FOV of 70 degrees horizontal and 60 degrees vertical. The Kinect draws approximately 2.67 A at 12 V.

B. *Wireless control Using ESP32*

The ESP32 microcontroller is used as the primary wireless correspondence to manually control the SmartStride. The board can be integrated through both Wi-Fi and Bluetooth connection. It provides an easy way of

transmitting user joystick commands to the Raspberry Pi. Any movement made with the joystick sends output voltage signals that are read through the ESP32's 12-bit ADC. The signals are processed at a baud rate of 115200 bps while the Esp32's clock operates at a frequency of 240 MHz [3]. The devices software constantly takes in user input, filters noise, and makes sure that the control data is sent at consistent intervals making for quick responsiveness and smooth wireless communication between the joystick and the wheelchairs microcontroller. (Raspberry Pi 5). The ESP32 Controller will be the main interface the user will control to access all the features of the wheelchair. Upon first initialization the user will be met with the main menu displaying all available functions the wheelchair is capable of. Exiting modes will be done by the Emergency Stop button being pressed. This also safely stops the wheelchair no matter what in case of the need for manual intervention. These functions include:

1) Mapping - Utilizes the Kinect sensor IR Flood Sensor to generate a 3D map by rotating the chair in a stationary position 360°.
2) Autocruise - Wheel Chair will cruise forward at a steady pace and stop promptly if any object obstructs its path. Once clear of any obstruction it will continue cruising.
3) Follow Person - Utilizing the Kinects Camera the wheelchair will follow the torso of the individual present in front of it at the time of menu selection.
4) Manual Control - Reverts to standard joystick operation, allowing the wheelchair to be driven manually, just as it was originally designed.

For wireless communication we are utilizing BLE(Bluetooth Low Energy) to maximize the battery life of the wireless controller. In tandem with this we also have the ESP32 act as the host of Bluetooth communication. Advertising tends to consume less energy than actively searching for a channel due to the ability to advertise blue tooth signals in pulses. Combining these practices helps maximize our battery life and ensure the user has a reliable controller that quickly accesses necessary functions.

C. *Central Processing Unit: Raspberry Pi 5*

The Pi serves as the primary processing unit within the project. It is responsible for making sure the wheelchair executes both the autonomous and manual control scripts. The Pi 5 has a 2.4 GHz quad-core ARM Cortex-A76 processor and can run up to 8 GB of RAM [4]. Under normal operating conditions, the Pi draws around 9.5 watts, which is super-efficient causing better battery life and enough power for real-time sensor signal processing and generation of control signals. For the autonomous driving mode, the Raspberry Pi processes incoming data from the Kinect sensor through its USB 3.0 plug-in. The Pi operates on a regulated 5 V DC input and communicates with the DAC and other peripherals via I2C. The versatile architecture makes for easy coordination of data for sensor input, wireless communication, and analog output making the Pi the heart of the project.

D. *Digital-To-Analog Conversion: MCP4728*

The MCP4728 has a critical role in the control system part of our project. It is the conversion pathway to get data from the Pi to the wheelchair's analog joystick inputs. The device has 4-channel, 12-bit DAC that receives data via I2C and outputs the relative analog voltage levels [5]. The data the DAC gets from the Pi, will then turn around and output 5V over 2 different channels at a time to the joystick. For example, if channel A (forward) gets 3V, and channel B (backwards gets) 2 V, then the chair will move forward slowly. The higher the difference between the two channels the DAC sends information through, the quicker, the speed of the wheelchair.

E. *Gyroscope & Magnetometer Module: MPU-9250*

The MPU9250 is a motion tracking device that will give us 3 axis gyroscopic data, magnetometer data, and acceleration data. The gyroscopic data and magnetometer data is most useful for us, so that we may use north as a point of truth during orientation and the gyroscope to know "how far" we have turned and in what direction. This chip will communicate to our Raspberry Pi via I2C, and is highly documented for integration.

VI. SOFTWARE DESIGN

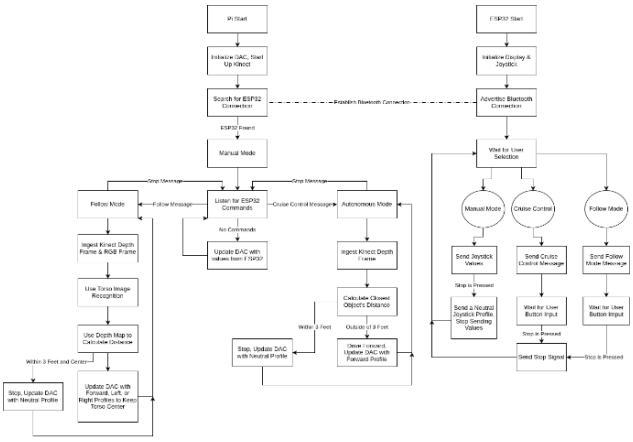

Fig. 6. Software flowchart showing SmartStride's dual-mode control logic.

The software architecture of the SmartStride performs under a two-mode control framework, enabling both manual and autonomous wheelchair operation. As shown in Fig. ******. Once the system powers on, the user can choose to either operate the wheelchair under Automatic Cruise or Manual control. In manual mode, the ESP32 processes joystick inputs and transmits them to the Raspberry Pi through Bluetooth. In autonomous mode, the Pi will make the chair go into a continuous forward driving mode while the Kinect depth sensor begins scanning and communicating what it sees with the Pi. When an object is detected, the wheelchair will stop, then execute a maneuver going around the object, then it will resume on its original path. This logic loop of detecting and avoiding obstacles will continue for the duration of automatic cruise mode. The flowchart highlights the software's simple yet efficient logic and a look at the sensor feedback making for a safe and reliable experience for the user.

A. Kinect v2 and libfreenect2

For the purposes of this design, robust and real-time environmental perception is the most critical requirement. This is accomplished using a Microsoft Kinect v2 sensor, which provides three essential data streams: a high-definition 1920x1080 color (RGB) stream, an infrared stream, and, most importantly for navigation, a 512x424 Time-of-Flight (ToF) depth map. This sensor provides dense 3D data of the environment in front of the wheelchair.

To interface this sensor with the system's onboard processing unit (an NVIDIA Jetson Nano), the open-source libfreenect2 library is utilized. libfreenect2 is a cross-platform driver that handles the high-bandwidth USB 3.0 communication protocol and data-packet processing required to access the Kinect's raw data streams [6].

The library's processing pipeline is essential to the project. It receives raw sensor data and performs registration, aligning the color image with the depth map. This allows the system to create a 3D point cloud where each point has both (X, Y, Z) spatial coordinates and (R, G, B) color information.

VII. TESTING AND RESULTS

A variety of tests were performed on the prototype SmartStride to make sure that it could safely and efficiently handle operations to be carried out by future users. The various tests done were: Mapping Ability Testing, Wireless Controller Test, Bluetooth Range Test, person detection, object detection, and average stopping distance

A. Mapping Ability Testing

For the mapping ability testing, the chair was put in autonomous mode, then the chair was told by a user code command input to start spinning in slow motion and to scan the room. After every 30 degrees of spinning the Depth Sensor takes a slice of its current view as an image and stores the data. Once the full 360 degree rotation is met, the chair stops spinning, then the data gets pulled together to output a depth map of all of the chairs current area. The experiment was done 3-times in different settings to make sure the resulting depth maps being made were consistent and accurate.

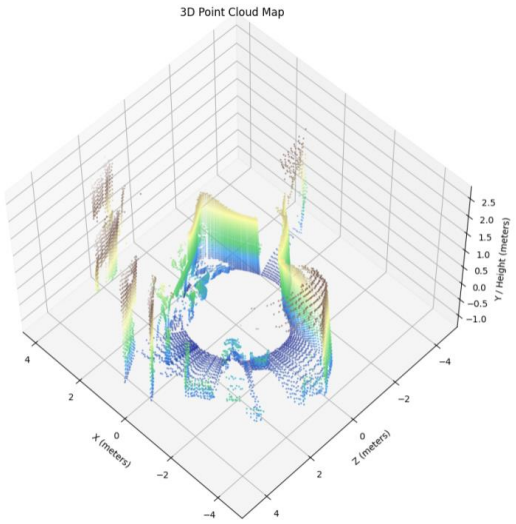

Fig. 7. 3D point cloud map based off of IR-Sensor data

B. Wireless Controller Testing

The controller board was turned on and connected to the SmartStride through a Bluetooth low energy link. It looks for the Pi located on the chair to make a connection. Once connected, the joystick starts sending signals so the chair can be controlled from a distance. The test was done multiple times to ensure quick and reliable wireless hookup.

C. Bluetooth Range Test

The controller board was hooked up to the chair from around 10 feet. The board was then walked in a straight line away from the wheelchair until the board disconnected from the wheelchair. During the test, the distance from the wheelchair at where the controller disconnected is what was measured. The test was done a total of 10 times. The average distance of disconnection was 23.73 feet and standard deviation of 1.49.

Table I. Average Bluetooth Range and Standard Deviation after 10 trials.

Trial Number	Distance (ft)
1	24.6
2	25
3	23.5
4	25
5	24.8
6	24
7	20
8	23.8
9	22.8
10	23.8

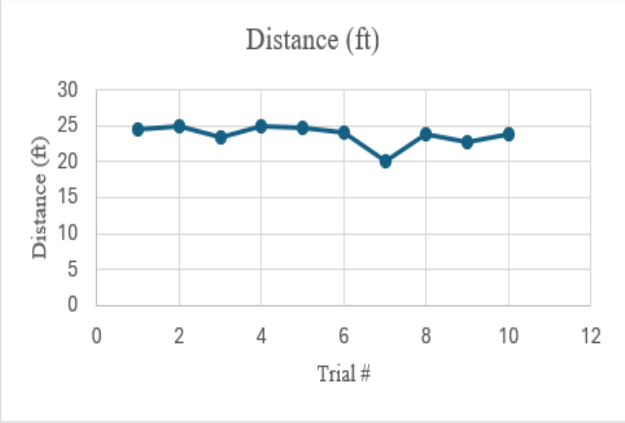

Fig. 8. Bluetooth range test results through 10 trials

D. *Person Detection test*

The wheelchair was turned on in autonomous mode and driven straight. A human test subject walked at a normal pace in front of the chair forcing the IR sensor readings to tell the chair to stop. This test was done 5 times, each at different speeds ranging from 2 to 6 MPH to ensure proper stopping reaction and user safety. Also, braking distance was set in the computer system to right at 4 feet The SmartStride successfully stopped each time. The results were successful braking even when the person moves from outside the field of view of the sensor.

E. *Object Detection and average stopping time*

A 3.5-ft tall object was placed about 8 feet in front of the smart stride. The smart stride on autonomous mode crept forward at a normal walking pace and was supposed to stop about 4-ft away from the object. The trial was conducted 10 times to see if the stopping distance was consistent. The results are shown in figure 10 and table III. Another trial was conducted during the same time to see about the average stopping reaction time at the same distance. The results are shown in figure 9 and table II.

Table II. Autonomous mode Reaction time results through 10 trials

Trials	Reaction Time by Trial (s)
1	0.33
2	0.66
3	0.476
4	0.577
5	0.267
6	0.447
7	0.789
8	0.554
9	0.632
10	0.212

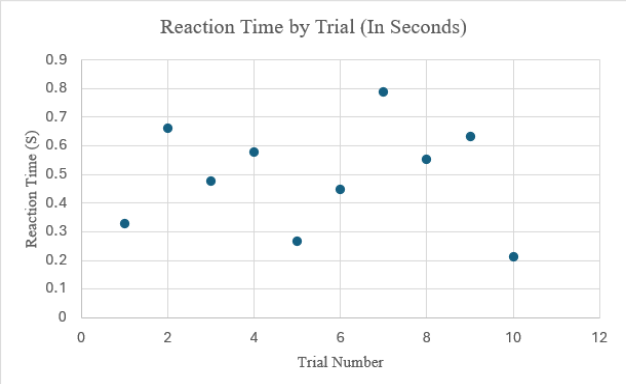

Fig. 9. Autonomous mode reaction time results through 10 trials.

Table III. Autonomous mode distance stopped from object results through 10 trials.

Trials	Distance Stopped (ft)
1	3.84
2	3.96
3	3.1
4	4.1
5	3.77
6	3.67
7	3.81
8	3.6
9	3.55
10	4.2

Fig. 10. Autonomous mode distance stopped from object results through 10 trials.

IX. CONCLUSION

The SmartStride Project successfully demonstrated an affordable, semi-autonomous wheelchair system that enhances mobility and user independence. Through integrating an ESP32, Raspberry Pi 5, and Kinect V2 sensor, the design achieved reliable autonomous and manual operation through in tune hardware and software control. The testing overall was a success showing the accuracy of how fast and efficient the wheelchair was at reacting to its surroundings, also proving to be a very safe ride.

THE TEAM

Matthew Itskovich is a graduating 24 year old Computer Engineering Student hoping to enter the world of embedded systems. Main interests include working PCB design and Soldering. He hopes to work in a big company like Intel or Nvidia to help design hardware. Main hobbies include video gaming and skateboarding.

Evan Rees is a graduating 23-year old Computer Engineering Major at University of Central Florida. He currently plans to work for Cole-Engineering as a Software Design Engineer here in Orlando. His current career goals are to work for a chip manufacturer like Intel or AMD as a Chip Designer. As a hobby he manages - a modest home lab for software experiments and simple testing

Adam Lilly is a graduating 24-year old Electrical Engineering Major who is hoping to pursue a career in construction consulting engineering. He plans on working towards getting his EIT and then PE. Main interests include watching college football and fishing

Arturo Lara is currently a senior studying at the University of Central Florida taking the College of Engineering's Computer Engineering Comprehensive Track Bachelor's degree with an expected graduation date of December 2025. He is employed at Phoenix Defense in UCF's Research Parkway in Configuration Management under the JLCCTC program. He plans to continue working as a manager after graduation and has an interest in computer programming and learning about embedded systems.567

ACKNOWLEDGEMENT

The authors would like to thank Dr. Chung Yong Chan and Dr. Arthur Weeks for their guidance and feedback

throughout the SmartStride project. Their support and mentorship were invaluable in shaping the direction, design process, and successful completion of this project.

REFERENCES

[1] J. Leaman and H. M. La, "A comprehensive review of smart wheelchairs: Past, present, and future," *IEEE Transactions on Human-Machine Systems*, vol. 47, no. 4, pp. 486–499, Aug. 2017, doi: 10.1109/THMS.2017.2706727.

[2] Advanced Monolithic Systems, *AMS1117: 1A Low Dropout Voltage Regulator Datasheet*, Rev. 2.0, AMS Inc., Sunnyvale, CA, USA, 2016. [Online]. Available: https://www.advanced-monolithic.com/pdf/ds1117.pdf

[3] Espressif Systems, *ESP32 Series Datasheet (v4.9)*, Espressif Systems, Shanghai, CN, Feb. 2025. [Online]. Available:https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf

[4] Raspberry Pi Ltd., *Raspberry Pi 5 Product Brief*, Revision 1.0, Sept. 2023. [Online]. Available:https://datasheets.raspberrypi.com/rpi5/raspberry-pi-5-product-brief.pdf

[5] Microchip Technology Inc., *MCP4728 – 12-Bit, Quad Digital-to-Analog Converter with EEPROM Memory (DS22187E)*, Chandler, AZ, USA: Microchip Technology Inc., 2010. [Online]. Available: https://ww1.microchip.com/downloads/en/DeviceDoc/22187E.pdf

[6]"OpenKinect/libfreenect2 – 1 Overview," DeepWiki, 2025. [Online]. Available: https://deepwiki.com/OpenKinect/libfreenect2/1-overview. [Accessed: Oct. 31, 2025].

[7] Texas Instruments, LM2596 Datasheet," 2017